CS4433
Software Engineering

Week 7 deliverable

Moderated Learning Management System

Lebohang Lebete | Student number: 202321241

Supervisor: Dr Mamoqenelo Morolong

National University of Lesotho

Continuation prepared from the supplied weeks 1-3. Weekly labels follow the seven-week assignment schedule. No historical review dates or signatures are inferred.

Week 7 - maintenance and completion

CR1 is implemented after the tagged v1.0 baseline. The consolidated report and viva guide complete the scheduled documentation work.

CR1 - class-wide extension (release v1.1)

Request: a lecturer can extend a deadline for a whole class, and students see extensions in their assignment list. Classification: adaptive/enhancement maintenance, primarily perfective, because it expands an already functioning individual-extension feature to a new school workflow rather than fixing a defect.

Approved-baseline impact

Artefact	Impact
FR11 / UC08	Retain individual extension; add FR11a / UC08a class extension
FR14 / UC12	Show effective deadline plus class and individual extension details
BR1 / UC20	effective = max(base, class extension, individual extension)
BR10 / DA1	Re-evaluate unmarked submissions; refuse class extension if any submission is already marked
Figure 5.1 components	No new process; existing ExtensionService and DAO gain operations
Figure 5.3 domain	Add ClassExtension associated 0..1 with Assignment and grantedBy User
Figures 5.4 / 5.5	grantClassExtension; ClassExtension query/command; shared deadline resolver
Figure 5.6 state	Class extension may move Late -> Submitted, like an individual extension
Figure 5.7 activity	Extension choice precedes deadline resolution; existing marking path unchanged
Figure 5.8 sequence	Resolve three deadline sources before submit/resubmit
Figure 5.9 sequence	Penalty recomputation uses shared resolver; no change after marking
Figure 5.10 / SQL	New class_extension table, assignment_id PK/FK, deadline, reason, actor, timestamp
Source	repository.hpp/.cpp, services.hpp/.cpp, server.cpp, client.cpp, schema.sql, migration-v1.1.sql
Tests	T15 precedence and visibility; T16 state, scope and atomic refusal; S21 HTTP visibility; all T01-T14 and S01-S20 rerun
Documentation	Student/lecturer manuals, installation migration, release notes, traceability and this supplement

FR11a: an assigned lecturer may set a class deadline later than the current class deadline, with a non-blank reason. FR14a: each enrolled student sees base deadline, optional class extension and individual extension reason/deadline, and effective deadline. These are Must requirements for CR1 and verified by T15/T16/S21.

The class extension applies to the assignment, not to a frozen list of enrolments; newly enrolled students therefore receive it. Individual extensions are preserved. A shorter class extension never takes away a longer individual accommodation. A later class extension supersedes an earlier class extension. No user sets the penalty directly. Marked work cannot acquire a new deadline retrospectively; the whole class operation is refused atomically if any submission is already marked. This preserves DA1 and avoids silently changing completed assessment snapshots. The school should review that policy if post-marking class extensions are required.

The new table is 3NF: assignment_id determines deadline, reason and actor/time. It avoids rewriting assignment.deadline and keeps the original baseline visible. Startup runs an idempotent CREATE TABLE IF

NOT EXISTS migration for existing v1.0 databases. Backup and restore automatically include the new table. The migration does not alter existing submissions, marks or audit entries. The upgrade test uses an actual v1.0 database, not only the fresh v1.1 schema.

Estimated change scope is one new persistent entity, two repository operations, one service method, one endpoint, one console action and two display fields. The release is isolated after tag v1.0; it does not modify that tagged baseline.

Executed evidence and acceptance limits

v1.0: 14 Catch2 cases, 219 assertions; 77 HTTP/system checks; business line coverage 95.77% (475/496 executable lines).

v1.1: 16 Catch2 cases; 81 HTTP/system checks; business line coverage 96.07% (513/534 executable lines). Actual v1.0 database upgrade and repeated migration checks passed. Cppcheck produced no diagnostics for the configured checks.

Load: 850 stored accounts, 50 concurrent clients. Login p95 2.5528 s (FAIL against 2 s), listing p95 0.0091 s (local PASS). All 800 submissions accepted in 44.342 s; 800 checksums verified; integrity ok. Workstation loopback only; target hardware, memory ceilings and physically disconnected external interface are not certified.

Human Gate 3, three-person usability trial and viva remain to be performed. GitHub remote publication and CI evidence remain blocked by read-only connector access and missing local Git authentication. The local bundle and push script are provided; no remote run is invented.

Learner identities, assessment files, marks and feedback require careful handling. Lesotho's Data Protection Act is published as Act 5 of 2012, with the source text's short title referring to the Data Protection Act, 2011. The school should confirm applicable legal obligations and commencement details with its responsible officer before operational use. The design addresses practical safeguards through limited role scope, deactivated accounts rather than deleted assessment identities, audit trails and restricted backups. These controls are relevant evidence of responsible processing, but do not establish legal compliance on their own. The institution still needs lawful processing procedures, retention rules and a way to handle access or correction requests.

The assessment setting makes integrity as significant as confidentiality. A mark must not change without an identifiable actor and a reason, and a student must not see an unmoderated value that may later be withdrawn. The server therefore owns all permission and penalty decisions. However, an operating-system administrator with direct database access is outside the application's audit boundary; append-only triggers are not a substitute for server access controls and secure backup handling. Plain HTTP also leaves credentials and marks observable to someone able to inspect LAN traffic. Isolation reduces exposure but does not remove it, so TLS remains a priority before use on a shared or untrusted network.

Fairness depends on making policy visible. The ceiling rule causes a one-second delay to count as one day late, and more than seventy-two hours produces zero even if the work is educationally valuable. The application consistently applies that stated rule and still accepts the work for feedback. Consistency alone does not prove fairness. Staff must communicate deadlines, use an accessible clock/time convention and offer an accountable extension process for illness, disability or disruption. CR1 helps respond consistently to school-wide disruption while preserving longer individual arrangements. Decisions after marking require an explicit policy change rather than an undocumented workaround or manual database edit.

Accessibility also needs evidence. A keyboard console removes dependence on a mouse and avoids some visual layout barriers, but structured JSON, technical IDs and UTC input can still confuse first-time users or screen-reader users. The manual, direct menu access and clear error messages are initial measures. Actual trials should examine reading order, language clarity, error recovery, contrast settings and whether users can

finish the core tasks independently. Finally, professional integrity requires the student to understand and explain this continuation, distinguish generated assistance from their own reviewed decisions as required by institutional policy, and report failed and unrun checks honestly during submission and viva.

12. Conclusion

The project now has a working, testable continuation from the approved week 3 design through the scheduled implementation, verification, deployment packaging and CR1 maintenance work. The evidence demonstrates role separation, server-controlled business rules, persistent file history, moderated release, audit integrity and a database-preserving upgrade. The weekly deliverables make that sequence reviewable without claiming that calendar weeks elapsed during this session. The remaining acceptance work is explicit: resolve or accept the measured login performance shortfall, run target-hardware and usability checks, complete supervisor Gate 3, authenticate publication and obtain a real hosted CI run. Those limitations are part of the engineering result, not omitted from it.

15-20 minute viva demonstration

Use synthetic accounts only. Start a clean instance and keep the test evidence open.

1. Explain the stakeholder conflict: lecturer immediate release versus principal

moderation. Demonstrate the hidden student result before HoD release.

2. Show the four polymorphic User subclasses and UserFactory; explain why a client

menu is not enough and demonstrate a server-side scope rejection.

3. Submit, resubmit and download a binary file. Show history, checksum and UTC receipt.

4. Mark a late submission; calculate a 50 maximum / 40 raw / 26-hour delay by hand:

two days, 10 marks penalty, 30 final. Explain the 72-hour-plus-one-second edge.

5. Return for re-marking, adjust a raw mark, approve, release and show audit old/new

values. Explain the transaction and optimistic version counter.

6. Show only Argon2id encoded hashes, never passwords. Explain salt, the two-hash

memory gate, lockout and transient session tokens.

7. Compare v1.0 and v1.1. Demonstrate a class extension and a longer individual

extension, then explain why class extensions after marking are rejected.

8. Run Catch2 and show coverage, the HTTP results, defect log and Cppcheck output.

Explain why 95% line coverage does not prove every business condition is correct.

9. State the measured login shortfall, unmeasured target hardware/memory/usability,

plain HTTP limitation and repository/read-pool design deviations candidly.

10. Show the weekly schedule, release tags and installation/recovery instructions.

Be ready to explain the single-value JSON binding defect, byte-safe download fix, rollback, why backup includes files, why no network share opens SQLite, and how the CR1 schema migrates an existing instance without changing released marks. The author must review and understand the supplied continuation before submission.

Updated risk register

ID	Risk	Exposure	Mitigation
R1	SQLite/file integrity	8	Transactions; serialised access; recovery and checksum tests; stopped-service backups
R2	Password implementation	5	libsodium Argon2id; unique salts; T10/S13; restricted server access
R3	Design time pressure	5	Design supplied and Gate 2 confirmed; preserve baseline; review deviations
R4	Scope expansion	6	Must requirements first; FR28 deferred; only scheduled CR1 added
R5	Insufficient business tests	3	Measured business coverage above 70%; retain boundary/role tests
R6	Single-developer continuity	15	Local Git bundle and packages; authenticated remote publication still needed
R7	Workload and understanding	6	Weekly work packages; viva rehearsal; student reviews all code
R8	Design/code mismatch	6	Explicit consolidated-DAO and serialised-read deviations; traceability supplement
R10	Hash queue exceeds login latency	16	Observed p95 2.5528s; preserve hashing; profile target hardware before acceptance
R11	Unencrypted HTTP LAN traffic	8	Loopback default; isolated LAN demonstration; TLS work pending
R12	Traceability loss	3	FR/UC/source/test matrix and CR1 impact table
R13	GitHub publication blocked	9	Read-only connector and no local sign-in; bundle and push script provided
R14	Usability/accessibility not observed	12	Three-user trial and supervisor Gate 3 remain pending

Week 7: actual C++ implementation

v1.1 source with CR1 class-wide extensions, effective-deadline display, migration, regression tests and upgrade evidence.

The complete source is supplied in `implementation/` and `implementation-week-7.zip`. This PDF also embeds that ZIP as a file attachment; open the Attachments pane in a compatible PDF reader to save it. The ordinary source folder and weekly delivery ZIP remain available if a PDF viewer cannot expose attachments.

Release v1.1: 16 Catch2 cases / 235 assertions; 81 HTTP/console checks; 96.07% business-logic coverage. Regression and actual v1.0 database migration evidence are included. Later usability update: 90 HTTP/console checks pass, including numbered menus for all four roles, role selection, yes/no validation and saved server-address lookup. The current source includes this update on top of the v1.1 baseline. Native Windows GUI update: lms_gui.exe (also packaged as School-LMS.exe) provides role navigation, dropdowns, file pickers and result tables. Validation and server errors preserve the current form and entries. All 32 native UI-component checks pass. Lecturer My courses and named record dropdowns are included; 17 core cases / 246 assertions pass, including course scoping and protected result data. Desktop visual verification was stopped with Escape; the automated form checks were completed.

Included file	Implementation responsibility
CMakeLists.txt	C++17 targets, dependencies, schema copying and CTest integration
database/migration-v1.1.sql	Idempotent class_extension upgrade for CR1
database/schema.sql	Persistent tables, constraints, indexes and audit triggers
include/lms/domain.hpp	Abstract User hierarchy, Factory contract, penalty Strategy, UTC and lifecycle contracts
include/lms/gui_actions.hpp	Supporting implementation source
include/lms/repository.hpp	DAO interface, fixed query/command identifiers and file-store abstraction
include/lms/services.hpp	Service interfaces and injected dependencies
src/client.cpp	Four-role interactive console, upload/download and API requests
src/desktop.manifest	Supporting implementation source
src/desktop.rc	Supporting implementation source
src/domain.cpp	Polymorphic factory, deadline/mark validation, late-penalty and lifecycle rules
src/gui.cpp	Supporting implementation source
src/repository.cpp	Parameterized SQLite statements, transactions, backups and file handling
src/server.cpp	HTTP endpoint registration, authentication and error translation
src/services.cpp	Role/scope enforcement and complete assessment business services
tests/coverage.py	Business-logic gcov measurement
tests/load_test.py	850-account / 800-submission synthetic workload
tests/system_test.py	Real HTTP workflow, faults, recovery and release console check
tests/tests.cpp	Catch2 unit/integration tests and v1.1 regression tests when applicable
tests/upgrade_test.py	Actual v1.0-created database upgrade verification

Build prerequisites: CMake 3.20+, C++17 compiler, Ninja and libsodium headers/library. Remaining source dependencies and licence notices are included.

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build build --parallel 2
ctest --test-dir build --output-on-failure
```

Implementation excerpts

These excerpts are navigational examples; the attached ZIP contains the complete source, not only the excerpts.

src/domain.cpp

```
PenaltyResult TieredDailyPenaltyPolicy::calculate(double raw, double maximum, int days) const {
    if (!std::isfinite(raw) || !std::isfinite(maximum) || maximum <= 0 || raw < 0 ||
        raw > maximum || days < 0)
        throw ValidationError("Mark must be within 0 and the positive maximum");
    double final = days > 3 ? 0 : std::max(0.0, raw - days * 0.1 * maximum);
    return {final, raw - final};
}
int TieredDailyPenaltyPolicy::daysLate(const std::string &submitted, const std::string &deadline) {
    auto difference = epoch(submitted) - epoch(deadline);
    return difference <= 0 ? 0 : static_cast<int>((difference + 86399) / 86400);
}
bool SubmissionLifecycle::canTransition(const std::string &from, const std::string &event,
                                        bool returned) {
    if (event == "resubmit")
        return from == "SUBMITTED";
    if (event == "extend")
        return from == "SUBMITTED" || from == "LATE";
    if (event == "mark")
        return from == "SUBMITTED" || from == "LATE" || from == "MARKED";
    if (event == "approve" || event == "return")
        return from == "MARKED" && !returned;
    if (event == "release")
        return from == "MODERATED";
```

src/services.cpp

```
void ExtensionService::grantClassExtension(const User &u, int id, const std::string &until,
                                           const std::string &reason) {
    requireText(reason, "Class extension reason");
    epoch(until);
    repo_.transaction([&] {
        auto a = assignmentScope(u, id, Action::Teach);
        std::string previous = a.at("deadline");
        auto existing = repo_.list(Query::ClassExtension, {id});
        if (!existing.empty())
            previous = existing[0].at("new_deadline");
        if (epoch(until) <= epoch(previous))
            throw ValidationError("Class deadline must move forward");
        auto submissions = repo_.list(Query::LecturerSubmissions, {id});
        for (const auto &s : submissions)
            SubmissionLifecycle::require(s.at("status"), "extend");
        repo_.execute(Command::ClassExtensionSave, {id, until, reason, u.id, clock_.now()});
        for (auto s : submissions) {
            auto effective = deadline(a, s.at("student_id"));
            repo_.execute(Command::ExtensionReclassify,
                          {epoch(s.at("submitted_at")) <= epoch(effective) ? "SUBMITTED" : "LATE",
                           s.at("id"), s.at("version")});
        }
    });
```

Validation limits: the measured login latency shortfall, pending human acceptance, target-hardware checks and GitHub authentication are recorded in the main report and README. Code in the source ZIP is organized by schedule scope; no historical weekly development dates are inferred.